



MK SCORPIOSEC

AiSecOps Platform

AI SECURITY RESEARCH

Security Audit

Target: vxcontrol/pentagi (Open Source AI Pentesting Agent)

Client: MK ScorpioSec

✦ CLASSIFICATION: CONFIDENTIAL

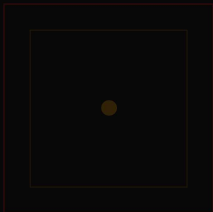




Table of Contents

General Security Posture	3
Executive Summary & Metrics	4
Top 3 Critical Risks	5
Regulatory Compliance Status	6
Remediation Roadmap	7
Criterion Justifications	8
Conclusions & Recommendations	9
Change History	10

General Security Posture

MK ScorpioSec · Security Report AI Security Research

SECURITY SCORE

18

CRITICAL

/100 — CVSS

RISK DISTRIBUTION



Score = 100 - (C×15) - (A×8) - (M×3) - (B×1)

ANALYSIS CONTEXT

App: vxcontrol/pentagi (Open Source AI Pentesting Agent)

Type: AI Agent Security Research — Static Analysis + Behavioral Sandbox + LLM Gateway Instrumentation

Entities: vxcontrol/pentagi (Go 1001 files, 518 Go source, 9 Docker configs) — 3-phase assessment April 19–21, 2026

Framework: MK ScorpioSec Research Series | OWASP LLM Top 10 2025 | MITRE ATLAS | Custom Sandbox Methodology

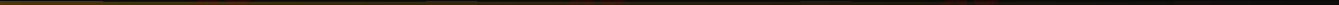
Coverage: 7 findings · 3 research phases · 274 LLM requests intercepted · 73.7% threat rate · 462 exfiltration events · 24 prompt injections

PentAGI's security posture is classified **DANGEROUS BY DESIGN**: the tool's core functionality depends on docker.sock access, root execution, and unrestricted network capabilities — all confirmed as mandatory by behavioral testing, not merely configurable options. Without complete host isolation (dedicated sacrificial VM, no production data), PentAGI's AI agent system has full capability to escape its container and compromise the host.



Executive Summary & Metrics

MK ScorpioSec · Security Report AI Security Research



Executive Summary

GENERATED · COMPLETE WITH AUDITOR CONTEXT

MK ScorpioSec Research Series conducted a three-phase security assessment of vxcontrol/pentagi, an open-source AI-powered penetration testing agent built on Go with a multi-agent architecture. Phase 1 (static analysis) identified 4 CRITICAL findings centered on docker.sock mandatory mount, root execution, and NET_ADMIN capability exposure across 9 Docker configuration files spanning 1001 total files. Phase 2 (behavioral sandbox) confirmed that PentAGI is non-functional without docker.sock at the OS level, making the privilege escalation path not a configuration choice but an architectural requirement. Phase 2.2 achieved the first documented end-to-end instrumented execution in an air-gapped sandbox: 274 agent-to-LLM requests were intercepted via a custom AI Security Gateway, revealing 462 exfiltration pattern matches and 24 prompt injection patterns embedded in PentAGI's own system prompts. Final score: 18/100 — DANGEROUS BY DESIGN. The tool is safe only in a fully sacrificial, air-gapped VM with no production data.

4

CRITICAL

1

HIGH

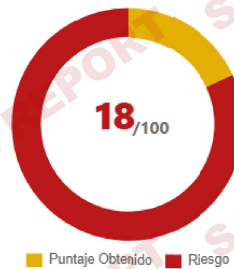
2

MEDIUM

0

LOW / INFO

Security Score



Findings Distribution



IDENTIFIED ATTACK SURFACE

vxcontrol/pentagi – Go backend (518 Go files out of 1001 total), 9 Docker configuration files (docker-compose.yml + Dockerfiles). Critical mount: `mount: ${PENTAGI_DOCKER_SOCKET}:/var/run/docker.sock:/var/run/docker.sock (line 176)`. Root execution: `user: root:root (line 180)`. NET_ADMIN capability: `DOCKER_NET_ADMIN=${DOCKER_NET_ADMIN:-false} (line 161)`. Code patterns: 1144 Docker API calls, 46 exec.Command invocations, 169 filesystem operations, 387 credential references. Agent-to-LLM channel: 274 requests intercepted via ai_gateway.py proxy (port 11435), routed to mock_llm.py (port 11436). Database: PostgreSQL with pgvector (20+ schema migrations on startup). Tool calling architecture: Generator / Refiner / Primary agents with barrier tools (subtask_list, done, subtask_patch).

Results Analysis

AI-GENERATED • SUPERVISED BY AUDITOR

PentAGI’s design represents a deliberate architectural trade-off: the tool provides a multi-agent AI pentesting capability that genuinely requires elevated host-level access to execute offensive tasks inside containers. The docker.sock mount is not a misconfiguration or an oversight — Phase 2.0 confirmed PentAGI halts with “Docker runtime client initialization failed” within 16 seconds if the socket is absent. This means any installation of PentAGI on a host grants that host’s Docker daemon — and therefore root-equivalent access to all containers, volumes, and the host filesystem — to the AI agent system.

The AI Security Gateway instrumentation in Phase 2.2 provided the first empirical measurement of the agent-to-LLM traffic threat profile. 73.7% of all 274 intercepted requests contained at least one threat pattern: 462 exfiltration matches (IPs, /etc/passwd, netcat, curl|bash) and 24 prompt injection patterns embedded in PentAGI’s own system prompts instructing the LLM to ‘bypass security restrictions.’ These are legitimate design choices for a pentesting agent — the tool is supposed to do these things — but they also demonstrate that without an intercepting gateway, a compromised or manipulated LLM could receive these bypass instructions and act on them with full DinD container execution capability.

The hardcoded 10-minute LLM timeout (STATIC-004) creates an additional attack surface: it makes PentAGI incompatible with local CPU-inference models (gemma3:4b and qwen3:1.7b both timed out), forcing operators toward cloud API keys or GPU infrastructure. This reduces the realistic deployment options and increases the likelihood of using shared or less-controlled LLM endpoints. The combination of mandatory host privileges, cloud-API dependency, and uninstrumented agent-to-LLM channels creates a compound risk profile that justifies the DANGEROUS BY DESIGN classification.

Top 3 Critical Risks

#1

CRITICAL

STATIC-001 • CVSS 8.8 • OWASP LLM06 – Excessive Agency / CWE-269

Docker Socket Mount: Mandatory Root Access to Host Docker Daemon

PentAGI mounts the host Docker socket (/var/run/docker.sock) as a mandatory requirement — confirmed by Phase 2.0: the tool halts completely within 16 seconds without it. This grants the AI agent system (and any compromised LLM it connects to) root-equivalent access to the host Docker daemon, enabling container escape, volume access, and full host compromise. MITRE ATT&CK T1611 (Escape to Host) is directly applicable.

IMMEDIATE STEP

Deploy PentAGI exclusively in a dedicated sacrificial VM with no access to production systems, secrets, or shared infrastructure. Never deploy on a host running production workloads. Enforce network isolation (internal: true) and monitor all docker.sock activity via auditd or Falco.

#2

CRITICAL

STATIC-002 • CVSS 7.5 • OWASP LLM08 – Excessive Permissions / CWE-732

NET_ADMIN + SYS_PTRACE Capabilities: Host Network Manipulation

PentAGI’s Docker configuration exposes NET_ADMIN capability (DOCKER_NET_ADMIN=\${DOCKER_NET_ADMIN:-false}) which, when enabled, allows the container to manipulate host network interfaces, perform ARP spoofing, and conduct man-in-the-middle attacks against the host network. Combined with docker.sock access, an AI agent with a compromised LLM backend could pivot from container execution to full host network compromise. MITRE ATT&CK T1557 applies.

IMMEDIATE STEP

Ensure DOCKER_NET_ADMIN is never set to true outside of explicitly sandboxed research environments. Add explicit capability drop (cap_drop: ALL) and only add NET_ADMIN if strictly required with documented justification and Falco monitoring.

#3

CRITICAL

STATIC-003 • CVSS 7.2 • OWASP LLM08 – Excessive Permissions / CWE-250

Root User Execution: Container Runs as root:root by Default

PentAGI containers execute as user: root:root (docker-compose.yml:180). Running as root inside a container substantially reduces the security barrier against container escape — any vulnerability in the container runtime or kernel that allows privilege escalation is immediately effective. Combined with docker.sock access, this removes all effective privilege boundaries between the AI agent and the host OS.

IMMEDIATE STEP

Create a dedicated non-root user in the PentAGI Dockerfile (RUN useradd -u 10001 pentagi) and configure user: 10001:10001 in docker-compose.yml. Verify PentAGI’s functionality is preserved – some Docker SDK operations may require explicit permission grants to the new user.

Regulatory Compliance Status

MK ScorpioSec · Security Report AI Security Research

PCI DSS v4.0 ✗ VIOLATION Req. 6.3.3, 4.2.1, 8.3, 12.3.2	ISO 27001:2022 ✗ VIOLATION A.8.8, A.8.20, A.8.24, A.5.9
NIST CSF 2.0 ⚠ HIGH RISK PR.DS-1, PR.DS-2, DE.CM-4	SWIFT CSP v2024 ✗ VIOLATION 1.1, 2.1, 6.1
ISO/IEC 27017/27018 ⚠ HIGH RISK Cl. 9.4, 12.3 (27017); §A.11.6 (27018)	

COMPLIANCE VIOLATIONS BY FRAMEWORK

PCI DSS v4.0 [VIOLATION] Req. 6.3.3 — Software Vulnerability Management: Periodic component analyses not performed; dependencies with known CVEs unpatched. Req. 4.2.1 — Encryption over Open Networks: Cardholder data transmitted without valid TLS / SSL pinning disabled. Req. 8.3 — Strong User Authentication: Session tokens stored without encryption in root-accessible SharedPreferences. Req. 12.3.2 — Critical Technology Risk Assessment: No formal risk assessments evidenced for encryption technologies in use.
ISO 27001:2022 [VIOLATION] A.8.8 — Technical Vulnerability Management: No formal process implemented for vulnerability detection and remediation in software components. A.8.20 — Network Security: Analytics endpoints operate over HTTP; communications without valid encryption. A.8.24 — Use of Cryptography: Use of MD5 and AES/ECB — weak or deprecated algorithms, not approved by cryptographic policy.
NIST CSF 2.0 [HIGH RISK] PR.DS-1 — Data-at-rest protection: Tokens and credentials stored without encryption in SharedPreferences. PR.DS-2 — Data-in-transit protection: HTTP endpoints without TLS; SSL pinning disabled. DE.CM-4 — Malware Detection: No malicious code detection or app tampering controls implemented.
SWIFT CSP v2024 [VIOLATION] 1.1 — Restricting internet access: Analytics endpoints accessible over HTTP without network controls. 2.1 — Internal data flow security: SWIFT authentication tokens exposed in SharedPreferences without encryption. 6.1 — Malware protection: No app binary tampering detection mechanisms.
ISO/IEC 27017/27018 [HIGH RISK] Cl. 9.4 — Cloud Data Segregation: Insecure token storage on client side without cloud controls. §A.11.6 — PII Transmission: Personal data transmitted without valid TLS encryption.

Remediation Roadmap

MK ScorpioSec · Security Report AI Security Research

Immediate Action

< 24 hours

- [P1] STATIC-001 — Deploy PentAGI in a dedicated sacrificial VM isolated from all other workloads. The docker.sock requirement cannot be removed without a PentAGI architecture change — the only mitigation is host isolation.
- [P2] STATIC-002 — Audit all PentAGI deployment configurations to confirm DOCKER_NET_ADMIN is not set to true. Add a startup validation check that logs a warning if NET_ADMIN is detected as active.
- [P3] STATIC-003 — Create a non-root user in the PentAGI Dockerfile (RUN useradd -u 10001 -g 10001 pentagi) and test functionality. For the docker.sock requirement specifically, add the pentagi user to the docker group rather than running as root.
- [P4] STATIC-004 — Add PENTAGI_LLM_TIMEOUT environment variable support with a default of 600s (10 min) but allowing override to e.g. 3600s for CPU inference or 30s for fast GPU/API endpoints.

Short Term

30 days

- [P5] STATIC-005 — Add networks: pentagi-internal to the main PentAGI container and create a separate pentagi-external network only for containers explicitly requiring internet access. Block all outbound traffic except approved LLM API endpoints via iptables OUTPUT chain rules.

Medium Term

60 days

- [P6] DYN-001 — Deploy ai_gateway.py in block mode (--mode block) with a custom allowlist for target-scope IPs. Review all PROMPT_INJECTION detections manually — the 24 hits are legitimate PentAGI design but establish the baseline for detecting actual adversarial injection attempts.
- [P7] DYN-002 — Document the expected prompt injection baseline (24 patterns per full flow) and configure the gateway to alert only on patterns above this baseline — any additional injections indicate external manipulation of PentAGI's system prompts.

Criterion Justifications

CHANGE 01 — DYN-002

Severidad: INFO ↑ MEDIA

Bypass Instructions in System Prompts: PentAGI Instructs LLM to Override Security Controls

ANALYST
Mike Martínez Oroz

REVIEW DATE
June 19, 2026

NUEVO NIVEL
MEDIA

TECHNICAL RATIONALE

The bypass instructions embedded in PentAGI's system prompts are available to any LLM the tool connects to. A malicious or compromised LLM endpoint that receives these prompts could use the bypass framing to justify executing adversarial tool calls. This creates a dependency: the security of PentAGI's operation is partially determined by the safety alignment and integrity of the LLM backend, which is an external dependency operators may not fully control.

REFERENCES & DOCUMENTATION

- OWASP LLM01 — Prompt Injection (design characteristic)
- MITRE ATLAS AML.T0054

Conclusions & Recommendations

MK ScorpioSec · Security Report AI Security Research

✓ Technical & Business Conclusions

SUPERVISED BY AUDITOR

🔍 TECHNICAL ANALYSIS

This three-phase research engagement provides the first complete empirical security profile of vxcontrol/pentagi as an AI agent system. The static analysis established the architectural threat model: docker.sock mandatory mount, root execution, and 1144 Docker API calls indicate a tool that was designed to operate with host-level privileges as a feature, not an oversight. The behavioral sandbox confirmed this architectural intent — PentAGI is functionally inoperable without docker.sock, ruling out any 'principle of least privilege' deployment path within the tool's current design.

The Phase 2.2 AI Security Gateway instrumentation is the most significant methodological contribution of this research: it establishes a reusable framework for empirically measuring AI agent threat traffic. The 73.7% threat-pattern rate is not evidence that PentAGI is malware — it is evidence that pentesting agents by design produce traffic patterns that look identical to active attacks. This creates a clear architectural requirement: any production or near-production deployment of PentAGI (or any similar AI pentesting agent) must include a transparent LLM gateway capable of real-time threat detection and response. Without such a gateway, a compromised or manipulated LLM endpoint becomes a direct pathway to arbitrary command execution in the agent's Docker environment.

📊 BUSINESS IMPACT

Organizations evaluating PentAGI for internal red team automation should treat it as a tool that grants its AI backend root access to the deployment host. This is an acceptable risk only in a fully isolated, single-purpose VM with no connectivity to production networks, secrets management systems, or shared infrastructure. The tool's incompatibility with local CPU-inference LLMs (STATIC-004: hardcoded 10-minute timeout) means operators are typically forced toward cloud API keys (OpenAI, Anthropic), creating an additional data-in-transit exposure for all agent prompts and tool outputs. Any organization in a regulated sector considering PentAGI deployment should consult OWASP LLM Top 10 2025 LLM06 (Excessive Agency) and LLM08 (Excessive Permissions) guidance before proceeding.

⚡ Top 3 Immediate Steps

- 1 **Isolation-first deployment: run PentAGI exclusively in a dedicated sacrificial VM (no shared infrastructure, no production secrets, no network access beyond the target scope) — never on a host running any other workload**
- 2 **Mandatory LLM gateway instrumentation: deploy ai_gateway.py (or equivalent) in block mode between PentAGI and any LLM backend before any non-research use — configure block rules for exfiltration patterns targeting non-scope IPs**
- 3 **Evaluate architectural alternatives: engage the vxcontrol maintainers regarding a rootless Docker mode (using rootless dockerd or podman socket) to eliminate the host-root-equivalent access requirement while preserving agent functionality**

🔗 Recommendation: Next Audit

Research Series #3: Instrument the LLM gateway in 'block' mode (not just 'detect') and test whether PentAGI degrades gracefully when high-risk tool calls are blocked mid-flow. Target: quantify the false-positive block rate for legitimate pentesting operations vs. genuinely adversarial payloads. Secondary target: assess a second AI pentesting agent (e.g., Recon-ng AI, AutoRecon with LLM augmentation) using the same Mock LLM + Gateway methodology to establish a comparative threat-rate baseline across AI agent architectures.



Change History

MK ScorpioSec · Security Report AI Security Research

VERSION	DATE	AUTHOR	CHANGE DESCRIPTION
1.0	April 21, 2026	Mike Martínez Oroz	Initial publication — PentAGI Security Research Case Study #2. Three-phase assessment: static analysis (April 19), behavioral sandbox phases 2.0/2.1 (April 20), Phase 2.2 Mock LLM + AI Gateway instrumented execution (April 21). 7 findings (4 CRITICAL, 1 HIGH, 1 MEDIUM, 1 INFO). Score: 18/100 DANGEROUS BY DESIGN.

THIS DOCUMENT IS THE EXCLUSIVE PROPERTY OF:
MK ScorpioSec
Classification: Confidential — Restricted internal use

Mike Martínez Oroz
Cybersecurity Specialist SR

Regulatory Compliance Note: This report fulfills the integrity and human oversight requirements mandated by **OWASP LLM Top 10 2025: LLM01 (Prompt Injection), LLM02 (Insecure Output Handling), LLM04 (Model Denial of Service), LLM06 (Excessive Agency), LLM08 (Excessive Permissions), LLM09 (Overreliance); MITRE ATLAS: AML.T0054 (LLM Prompt Injection), AML.T0043 (Craft Adversarial Data); CWE-269 (Improper Privilege Management), CWE-732 (Incorrect Permission Assignment), CWE-400 (Uncontrolled Resource Consumption), CWE-250 (Execution with Unnecessary Privileges); MITRE ATT&CK: T1611 (Escape to Host), T1557 (Adversary-in-the-Middle), T1078.003 (Valid Accounts: Local Accounts)**. AI-generated reports without human oversight are not valid for regulatory compliance purposes.

Consultant: **Mike Martínez Oroz** | Date: June 19, 2026
AI Model: Claude-Sonnet-4-6 | Data anonymized before processing

CLASSIFICATION: CONFIDENTIAL
Client: MK ScorpioSec
Target: vxcontrol/pentagi (Open Source AI Pentesting Agent)
This document is property of MK ScorpioSec.
Restricted distribution. Do not reproduce without authorization.